

Estudo de Caso – Sistema de E-commerce: Da Operação ao Relatório Analítico

Contexto

Você foi contratado por uma empresa fictícia de e-commerce chamada **ShopData**, que deseja integrar seu sistema de vendas (OLTP) com um ambiente analítico (OLAP) para geração de relatórios e dashboards de gestão.

Atualmente, o sistema transacional (OLTP) já registra pedidos, produtos e clientes em um banco PostgreSQL, mas a empresa precisa de uma **estrutura analítica simplificada**, onde seja possível consultar indicadores como:

- **Total de vendas por mês**
- **Produtos mais vendidos**
- **Gênero de clientes com maior volume de compras**
- **Ticket médio mensal**

Sua missão é criar o **schema analítico** e preparar uma **tabela agregada** com os dados consolidados, pronta para integração com ferramentas de BI.

Objetivo Geral

Implementar, no PostgreSQL, um modelo analítico derivado do sistema transacional existente

Utilizando **schemas, agregações, views materializadas e procedures** para simular um processo de atualização automática.

Objetivos Específicos



Criar o schema analítico



Criar tabelas de dimensão e fato no novo schema



Agregar dados de vendas por período, produto e cliente



Criar uma view materializada com o resultado consolidado



Desenvolver uma procedure que atualize automaticamente os dados agregados



Simular a execução da procedure e gerar um relatório simples

Estrutura do Estudo de Caso

Parte 1 – Preparando o ambiente (schema OLTP)

1. Crie um schema chamado loja (caso não exista).

```
CREATE SCHEMA loja;  
SET search_path TO loja;
```

2. Crie as tabelas abaixo e insira alguns registros de exemplo.

```
CREATE TABLE loja.cliente (  
  id_cliente SERIAL PRIMARY KEY,  
  nome VARCHAR(100),  
  genero CHAR(1),  
  cidade VARCHAR(50)  
);  
  
CREATE TABLE loja.produto (  
  id_produto SERIAL PRIMARY KEY,  
  nome VARCHAR(100),  
  categoria VARCHAR(50),  
  preco NUMERIC(10,2)  
);  
  
CREATE TABLE loja.pedido (  
  id_pedido SERIAL PRIMARY KEY,  
  id_cliente INT REFERENCES loja.cliente(id_cliente),  
  data_pedido DATE,  
  valor_total NUMERIC(10,2)  
);  
  
CREATE TABLE loja.item_pedido (  
  id_item SERIAL PRIMARY KEY,  
  id_pedido INT REFERENCES loja.pedido(id_pedido),  
  id_produto INT REFERENCES loja.produto(id_produto),  
  quantidade INT,  
  valor_total NUMERIC(10,2)  
);
```

3. Insira registros fictícios (pelo menos 5 clientes, 5 produtos e 10 pedidos com diferentes datas e valores).



Parte 2 – Criando o schema analítico (OLAP)

1. Crie o schema analítico

```
CREATE SCHEMA analitico;
```

Este schema será dedicado exclusivamente aos dados analíticos, separando claramente as operações transacionais das consultas de BI.

2. Crie as tabelas de dimensão e a tabela fato

```
CREATE TABLE analitico.dim_cliente AS  
SELECT id_cliente, nome, genero, cidade  
FROM loja.cliente;
```

```
CREATE TABLE analitico.dim_produto AS  
SELECT id_produto, nome, categoria  
FROM loja.produto;
```

```
CREATE TABLE analitico.fato_vendas (  
  id_pedido INT,  
  id_cliente INT,  
  id_produto INT,  
  data_pedido DATE,  
  valor NUMERIC(10,2)  
);
```

Parte 3 – Inserindo dados agregados na tabela fato

1. Popule a tabela fato_vendas com base nas tabelas do schema loja.

```
INSERT INTO analitico.fato_vendas
(id_pedido, id_cliente, id_produto, data_pedido, valor)
SELECT
p.id_pedido,
p.id_cliente,
i.id_produto,
p.data_pedido,
i.valor_total
FROM loja.pedido p
JOIN loja.item_pedido i ON i.id_pedido = p.id_pedido;
```

2. Verifique se os dados foram inseridos corretamente.

```
SELECT * FROM analitico.fato_vendas LIMIT 10;
```

 **Importante:** A tabela fato é o coração do modelo analítico, conectando todas as dimensões e armazenando as métricas de negócio.



Parte 4 – Criando a View Materializada

Crie uma **view materializada** que consolide as vendas por mês, produto e gênero do cliente.

```
CREATE MATERIALIZED VIEW analitico.vendas_resumo AS
SELECT
    DATE_TRUNC('month', f.data_pedido) AS mes,
    dp.categoria,
    dc.genero,
    SUM(f.valor) AS total_vendas,
    COUNT(f.id_pedido) AS qtd_pedidos,
    ROUND(AVG(f.valor),2) AS ticket_medio
FROM analitico.fato_vendas f
JOIN analitico.dim_produto dp
    ON dp.id_produto = f.id_produto
JOIN analitico.dim_cliente dc
    ON dc.id_cliente = f.id_cliente
GROUP BY mes, dp.categoria, dc.genero
ORDER BY mes;
```

Verifique os dados:

```
SELECT * FROM analitico.vendas_resumo;
```

Parte 5 – Criando uma Procedure para atualizar a View

Crie uma procedure para atualizar a view materializada sempre que novos pedidos forem registrados.

```
CREATE OR REPLACE PROCEDURE analitico.atualizar_vendas_resumo()  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    RAISE NOTICE 'Atualizando dados agregados...';  
    REFRESH MATERIALIZED VIEW analitico.vendas_resumo;  
    RAISE NOTICE 'Atualização concluída.';  
END;  
$$;
```

Execute a procedure:

```
CALL analitico.atualizar_vendas_resumo();
```

Automação

A procedure permite automatizar a atualização dos dados analíticos, garantindo que os relatórios estejam sempre atualizados.

Performance

Views materializadas armazenam os resultados pré-calculados, acelerando drasticamente as consultas analíticas.

Parte 6 – Gerando o relatório final

Crie uma consulta para gerar um relatório analítico agrupado por mês e categoria de produto:

```
SELECT  
  TO_CHAR(mes, 'YYYY-MM') AS mes,  
  categoria,  
  genero,  
  total_vendas,  
  qtd_pedidos,  
  ticket_medio  
FROM analitico.vendas_resumo  
ORDER BY mes, categoria;
```



Este relatório fornece uma visão consolidada das vendas, permitindo análises rápidas sobre o desempenho do negócio por diferentes dimensões.

Desafio Final

01

Insira novos pedidos no schema loja (novos clientes e produtos)

03

Gere novamente o relatório e observe as mudanças

Questões para reflexão:

- Como o modelo analítico ajuda a reduzir o tempo das consultas?
- Em que situações o modelo OLAP seria mais vantajoso que o OLTP?

02

Atualize a view materializada com a procedure

04

Reflita sobre os resultados

Conclusão

Com esse exercício, você simulou a transformação de um banco operacional (OLTP) em um ambiente analítico (OLAP)

Esse é o mesmo processo usado em projetos reais de **Business Intelligence (BI)** para alimentar relatórios em ferramentas como Power BI, Metabase ou Tableau.